

WPRO: Future-Aware Online Orchestration for Agentic LLM Workflows

Anonymous Authors - INFOCOM-style paper draft

Abstract- Agentic AI services are changing cloud LLM serving from request-level inference into workflow-level orchestration. A single request may trigger planning, retrieval, reasoning, generation, verification, and repair stages. Scheduling therefore determines not only which stage runs next, but also which models will be needed next and which models remain resident on GPUs. We formulate this future-coupled orchestration problem as an event-driven SMDP and propose WPRO, a workflow-progress and residency-aware actor-critic framework.

1 Introduction

AIaaS platforms increasingly execute agentic workflows rather than isolated LLM calls. Deep research assistants, coding agents, and document-analysis services decompose one request into dependent semantic stages. This makes the serving unit a workflow DAG.

The key challenge is future coupling. Completing a stage releases downstream stages, while selecting a model changes GPU model residency. A myopic dispatch may reduce immediate latency but evict a model that many soon-to-be-ready stages require.

WPRO addresses this gap with a future-aware online orchestrator that combines workflow-progress representation, DAG-induced model-demand estimation, residency-aware action scoring, hierarchical autoregressive dispatch, WAIT decisions, and time-aware actor-critic learning.

The paper is organized around one claim: in agentic AI workflows, scheduling no longer determines only which stage runs next; it also shapes which models will be needed next. This is the main reason why request-level LLM serving abstractions and static workflow schedulers are insufficient.

The intended contribution is not a lower-level replacement for vLLM, Orca, or Sarathi-Serve. Instead, WPRO is a higher-level workflow orchestrator that can sit above such runtimes and decide which workflow-stage-model-GPU action should be issued at each event.

2 Related Work

LLM serving systems such as vLLM, Orca, Sarathi-Serve, DistServe, and Splitwise optimize token-level execution, batching, or prefill/decode scheduling. Multi-adapter systems such as S-LoRA and Punica optimize adapter residency.

Classical workflow and cluster schedulers address DAG execution, deadlines, or resource placement, while RL schedulers learn policies over state snapshots. WPRO differs by explicitly modeling workflow evolution and future model-residency demand in a unified online policy.

Agentic LLM systems such as ReAct, Toolformer, Reflexion, AutoGen, and SWE-agent motivate the workload model because they turn one user request into multiple reasoning, tool, and verification stages. Their focus is agent capability; our focus is multi-tenant platform orchestration.

The baseline discussion in the TeX draft is intentionally careful: vLLM-, Orca-, and Sarathi-inspired policies are adapted under a unified simulator unless the full runtime is integrated. This avoids an overclaim that reviewers could attack.

3 Modeling and Formulation

At event k , the system state is $S_k = (W_k, R_k, G_k, M_k, t_k)$, where W_k is the active workflow set, R_k is the ready LLM-stage set, G_k is

GPU state, M_k is model residency, and t_k is the event time.

A dispatch action chooses a tuple (j, i, m, g) or $WAIT_g$. Feasible actions satisfy workflow precedence, model capability, GPU memory, and no-duplicate-stage constraints. Tool stages run outside the GPU queue and create completion events.

The objective maximizes weighted SLA-compliant value: $\sum_j w_j I\{T_{j-a_j} \leq D_{j-a_j}\}$. We also report weighted goodput rate, admission ratio, on-time ratio, P95 latency, residency hit, preparation time, and decision overhead.

Stage execution is divided into preparation, token execution, and communication. Preparation depends on whether the target model is resident, adapter-compatible, or cold-loaded. Token execution depends on input tokens, output tokens, model/GPU rates, and semantic stage type.

The simulator separates expected execution time from sampled execution time. Policies score candidates using deterministic expected latency, while actual perturbation is sampled only after a dispatch is committed. This keeps baselines fair under the same random seed.

The offline problem is already NP-hard under a one-GPU, one-event, independent-stage restriction by reduction from 0-1 knapsack. The online problem is harder because workflow arrivals, tool completion, communication delay, and residency are event-driven.

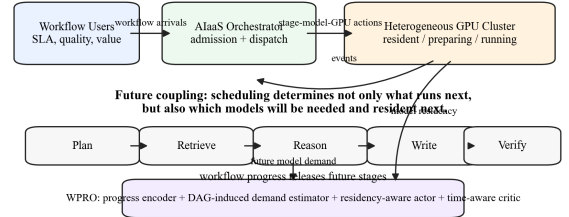


Fig. 1. Future coupling in WPRO: workflow progress releases future stages, while model selection changes GPU residency.

4 Online Orchestration Design

Workflow-progress representation

WPRO encodes completed stages, ready stages, remaining critical path, successor-stage types, slack, queue waiting, and workflow weight. Active workflows and GPUs are pooled with permutation-invariant encoders.

Future model-demand prediction

The auxiliary head estimates DAG-induced near-future demand $d_m^{\wedge} DAG(H)$ from unfinished workflows. It does not claim perfect prediction of unknown future arrivals.

Residency-aware action scoring

Each candidate action has action-specific features $\phi(S, a)$, including workflow, stage, model, GPU, preparation, execution, resident-hit, and evicted-model demand features.

Hierarchical autoregressive policy

WPRO processes idle GPUs sequentially, updates action masks after each selection, and includes WAIT actions to preserve valuable residency until the next exogenous event.

Time-aware actor-critic learning

Because event intervals are irregular, WPRO uses the SMDP discount $\exp(-\beta \Delta t_k)$ in TD targets and GAE.

Potential shaping

The progress potential increases with completed workflow progress and decreases when slack is consumed without progress. This prevents the policy from receiving positive shaping reward merely by waiting closer to a deadline.

Component	Role
Progress Encoder	Captures workflow evolution and remaining critical path
Demand Estimator	Estimates future model pressure from unfinished DAGs
Residency Scorer	Pricing target-model demand, evicted-model demand, and prep cost
Autoregressive Decoder	Builds feasible multi-GPU dispatch without full matrix enumeration
Time-aware Critic	Uses event interval in SMDP value learning

5 Analysis

The offline problem is NP-hard even for independent single-stage workflows on one GPU with a common deadline, by reduction from 0-1 knapsack. Online orchestration is harder because arrivals, tool completions, and execution perturbations are event-driven.

The autoregressive decoder is feasible by construction: it masks unready stages, infeasible models, memory violations, and stages already selected at the event. Its per-event candidate scoring complexity is $O(|I_k||R_k|M_{\max}F)$, avoiding exponential dispatch-matrix enumeration.

Potential-based shaping uses the same event discount as the SMDP return. Therefore it reshapes learning signals without changing the optimal policy class. This point is important because WAIT is useful only if waiting itself is not accidentally rewarded.

The Bellman coupling explains why greedy dispatch can fail. The same action changes both the future ready-stage set and future model preparation cost. A one-step score cannot evaluate this downstream effect unless it contains an explicit surrogate for future model demand.

6 Performance Evaluation

The simulator separates LLM, tool, and communication stages; models heterogeneous planning, reasoning, generation, retrieval, and verification capabilities; and uses chronological train/validation/test trace splits. Baselines include FCFS, EDF, SRPT, Utility-Greedy, Lyapunov, DAG-Oracle Greedy, Vanilla A2C, PPO, and runtime-inspired policies.

The final INFOCOM protocol should use five independent RL training seeds, validation checkpoint selection, at least twenty paired held-out test windows, bootstrap 95% confidence intervals, and Wilcoxon signed-rank tests against the strongest online baseline.

The main metrics are weighted completed value V_w , weighted goodput rate G_w , admission ratio R_{adm} , on-time ratio R_{on} , conditional SLA success, P95 latency, residency hit ratio, full-load count, queue waiting, communication overhead, and decision overhead.

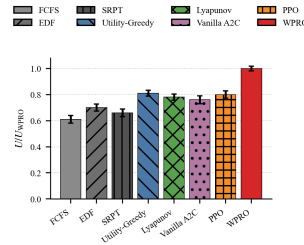


Fig. 2(a). Weighted utility.

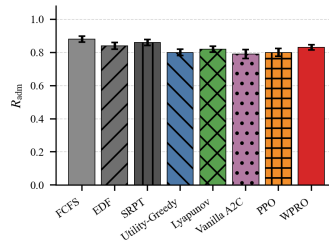


Fig. 2(b). Admission ratio.

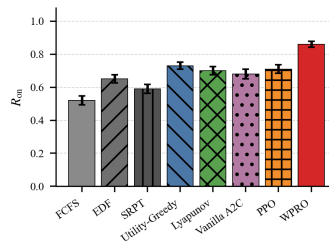


Fig. 2(c). On-time completion.

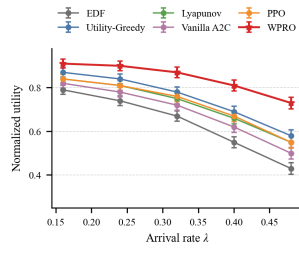


Fig. 3(a). Utility under increasing arrival rate.

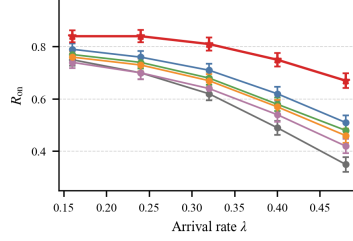


Fig. 3(b). On-time ratio under increasing arrival rate.

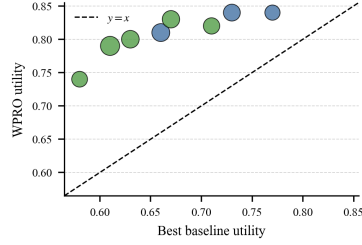


Fig. 4. Multi-environment improvement over the strongest baseline.

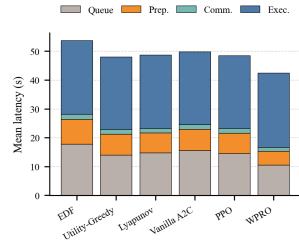


Fig. 5. Latency breakdown.

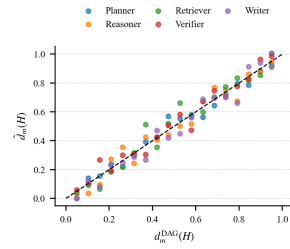


Fig. 6(a). DAG-induced demand estimation.

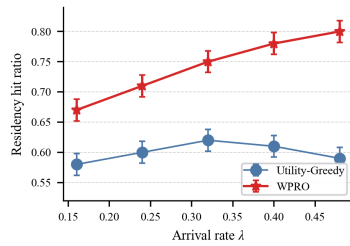


Fig. 6(b). Residency hit ratio.

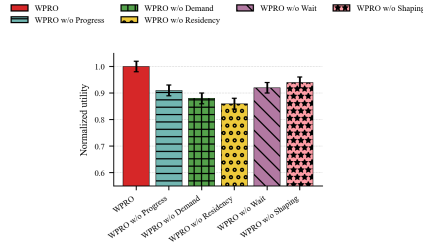


Fig. 7(a). Ablation on utility.

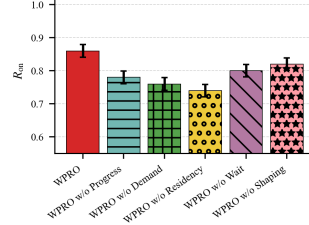


Fig. 7(b). Ablation on deadline satisfaction.

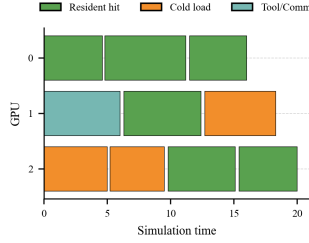


Fig. 8(a). Event timeline.

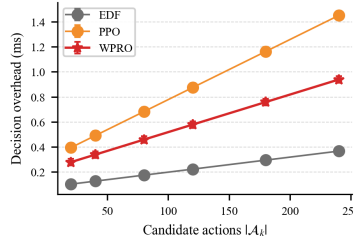


Fig. 8(b). Decision overhead.

Overall results should be interpreted through the service objective rather than a single utilization metric. WPRO may not maximize admission ratio in all settings, because admitting too many workflows can reduce on-time completion. Its advantage is stronger weighted SLA-compliant goodput.

The scalability experiment tests the core future-aware claim. Under light load, many policies have enough slack to recover from bad residency choices. Under heavy load, cold-load mistakes accumulate, and WPRO's demand and residency modules become more valuable.

The ablation study links performance to algorithmic structure. Removing future demand weakens anticipation of downstream model pressure; removing residency features weakens model-retention decisions; removing WAIT prevents the policy from preserving valuable resident models.

7 Discussion and Conclusion

WPRO is a workflow-level orchestrator and is complementary to token-level systems such as vLLM, Orca, and Sarathi-Serve. Runtime-inspired baselines should be described as adapted policies unless full systems are integrated.

The demand predictor estimates DAG-induced near-future model demand from active workflows rather than unknown future arrivals. Future work can add richer conditional branches, retry behavior, and production runtime integration.

WPRO shows that future-aware orchestration is essential for agentic LLM services: stage dispatch and model residency must be optimized jointly to improve weighted SLA-compliant goodput.

The final paper should keep the claims tight: WPRO does not claim perfect future prediction and does not claim full reproduction of every LLM runtime. It claims that exposing workflow progress and residency-aware future demand to the policy improves online orchestration in a unified, controlled environment.

References

- [1] Kwon, Woosuk and Li, Zhuohan and Zhuang, Siyuan and Sheng, Ying and Zhen. *vLLM*. 2023.
- [2] Yu, Gyeong-In and Jeong, Joo Seong and Kim, Geon-Woo and Kim, Soojeong a. *Orca: A Distributed Serving System for Transformer-Based Generative Models*. 2022.
- [3] Agrawal, Amey and Panwar, Ashish and Mohan, Jayashree and Kwatra, Nipun . *Sarathi-Serve: Efficient LLM*. 2024.
- [4] Zhong, Yinmin and Liu, Shengyu and Chen, Junda and Hu, Jiaqi and Zhu, Yi. *DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving*. 2024.
- [5] Sheng, Ying and Cao, Shiyi and Li, Dacheng and Hooper, Coleman and Lee, . *S-LoRA*. 2024.
- [6] Chen, Lequn and Ye, Zhenyu and Wu, Yao and Zhuo, Danyang and Ceze, Luis . *Punica: Multi-Tenant LoRA*. 2024.
- [7] Dao, Tri and Fu, Daniel Y. and Ermon, Stefano and Rudra, Atri and R{\'e}. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO*. 2022.
- [8] Pope, Reiner and Douglas, Sholto and Chowdhery, Aakanksha and Devlin, Ja. *Efficiently Scaling Transformer Inference*. 2023.
- [9] Miao, Xupeng and Oliaro, Gabriele and Zhang, Zhihao and Cheng, Xinhao an. *SpecInfer*. 2023.
- [10] Sheng, Ying and Zheng, Lianmin and Yuan, Binhang and Li, Zhuohan and Rya. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU*. 2023.
- [11] Aminabadi, Reza Yazdani and Rajbhandari, Samyam and Zhang, Minjia and Aw. *DeepSpeed*. 2022.
- [12] Patel, Pratyush and Choukse, Esha and Zhang, Chaojie and Shah, Aashaka a. *Splitwise: Efficient Generative LLM*. 2024.
- [13] Sun, Biao and Huang, Zhenyu and Zhao, Huanle and Hu, Hao and Li, Conglon. *Llumnix: Dynamic Scheduling for Large Language Model Serving*. 2024.
- [14] Wang, Yuxuan and Chen, Yiming and Li, Zhenyu and Sun, Lichao and Wang, X. *BurstGPT*. 2024.
- [15] Brown, Tom B. and Mann, Benjamin and Ryder, Nick and Subbiah, Melanie an. *Language Models are Few-Shot Learners*. 2020.
- [16] Bommasani, Rishi and Hudson, Drew A. and Adeli, Ehsan and Altman, Russ a. *On the Opportunities and Risks of Foundation Models*. 2021.
- [17] {OpenAI. *GPT-4*. 2023.
- [18] Ouyang, Long and Wu, Jeffrey and Jiang, Xu and Almeida, Diogo and Wainwr. *Training Language Models to Follow Instructions with Human Feedback*. 2022.
- [19] Wei, Jason and Wang, Xuezhi and Schuurmans, Dale and Bosma, Maarten and . *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022.
- [20] Yao, Shunyu and Zhao, Jeffrey and Yu, Dian and Du, Nan and Shafran, Izha. *ReAct*. 2023.
- [21] Schick, Timo and Dwivedi-Yu, Jane and Dess{\`i}. *Toolformer: Language Models Can Teach Themselves to Use Tools*. 2023.
- [22] Shinn, Noah and Cassano, Federico and Gopinath, Ashwin and Narasimhan, K. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023.
- [23] Wu, Qingyun and Bansal, Gagan and Zhang, Jieyu and Wu, Yiran and Li, Bei. *AutoGen*. 2023.
- [24] Yao, Shunyu and Yu, Dian and Zhao, Jeffrey and Shafran, Izhak and Griffi. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. 2023.
- [25] Yang, John and Jimenez, Carlos E. and Wettig, Alexander and Lieret, Kili. *SWE-agent*. 2024.
- [26] Qian, Chen and Cong, Xin and Yang, Cheng and Chen, Weize and Su, Yusheng. *Communicative Agents for Software Development*. 2023.
- [27] Topcuoglu, Haluk and Hariri, Salim and Wu, Min-You. *Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing*. 2002.
- [28] Mao, Hongzi and Alizadeh, Mohammad and Menache, Ishai and Kandula, Srika. *Resource Management with Deep Reinforcement Learning*. 2016.
- [29] Mao, Hongzi and Schwarzkopf, Malte and Venkatakrishnan, Shaileshh Bojja . *Learning Scheduling Algorithms for Data Processing Clusters*. 2019.
- [30] Gog, Ionel and Schwarzkopf, Malte and Gleave, Adam and Watson, Robert N.. *Firmament: Fast, Centralized Cluster Scheduling at Scale*. 2016.
- [31] Ousterhout, Kay and Wendell, Patrick and Zaharia, Matei and Stoica, Ion. *Sparrow: Distributed, Low Latency Scheduling*. 2013.
- [32] Gu, Juncheng and Chowdhury, Mosharaf and Shin, Kang G. and Zhu, Yibo and. *Tiresias: A GPU*. 2019.
- [33] Xiao, Wencong and Ren, Sijie and Li, Yong and Zhang, Yangqing and Hou, P. *Gandiva: Introspective Cluster Scheduling for Deep Learning*. 2018.
- [34] Liu, C. L. and Layland, James W.. *Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment*. 1973.
- [35] Sutton, Richard S. and Barto, Andrew G.. *Reinforcement Learning: An Introduction*. 2018.
- [36] Mnih, Volodymyr and Badia, Adri{\`a}. *Asynchronous Methods for Deep Reinforcement Learning*. 2016.
- [37] Schulman, John and Moritz, Philipp and Levine, Sergey and Jordan, Michae. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2016.
- [38] Schulman, John and Wolski, Filip and Dhariwal, Prafulla and Radford, Ale. *Proximal Policy Optimization Algorithms*. 2017.
- [39] Puterman, Martin L.. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. 1994.